

* object detection *

Object Detection(객체 탐지)란? 이미지나 동영상에서 객체(Object)의 위치(Bounding Box)와 종류(Class)를 동시에 탐지하는 기술이다. 단순히 객체가 있는지 여부를 판단하는 이미지 분류(Classification)와 달리, 객체의 위치(좌표)와 종류를 함께 예측한다.

(^人^) Object Detection 주요 개념

1) Bounding Box(바운딩 박스)

객체가 위치한 영역을 감싸는 사각형 좌표 (x, y, w, h)

x, y = 좌상단 좌표, w, h = 박스의 너비와 높이

Bounding Box란 객체가 위치한 부분을 감싸는 최소한의 사각형 영역이다. 이 박스를 통해 우리가 객체의 위치를 알 수 있고 크기와 비율을 파악할 수 있으며 그 정보를 바탕으로 라벨링, 추적, 분류 등을 수행할 수 있다.

✅ Bounding Box 형식 종류

① (x, y, w, h) 형식 — COCO 포맷

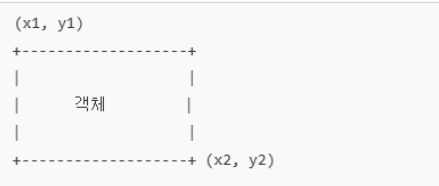
x, y = 좌상단 좌표
w, h = 박스의 너비, 높이



예: [100, 150, 60, 80] → x=100, y=150, w=60, h=80

② (x1, y1, x2, y2) 형식 - YOLOv8, OpenCV 등에서 자주 사용

(x1, y1): 좌상단 좌표
(x2, y2): 우하단 좌표



이 경우 너비와 높이는 계산해야 해요:

$$w = x2 - x1 \quad h = y2 - y1$$

③ YOLO 포맷 (중심 좌표 기반)

YOLO에서는 학습 시 상대 좌표 중심 중심 포맷을 사용

[x_center, y_center, width, height]
→ 모든 값은 0~1 사이 (이미지 크기에 대한 비율)

예: [0.5, 0.5, 0.3, 0.4] → 이미지의 가운데 박스

💡 정리

형식	설명	예시
(x, y, w, h)	좌상단 기준, 너비/높이 포함	COCO, Pascal VOC
(x1, y1, x2, y2)	양끝 좌표	YOLO 출력, OpenCV
(cx, cy, w, h)	중심 좌표 기반 (비율)	YOLO 학습용

COCO (Common Objects in Context)는 객체 감지, 분할(Segmentation), 키포인트 탐지 등에서 사용되는 표준 데이터셋 포맷이다. 보통 JSON 파일로 객체들의 주석(annotation) 정보를 담고 있다.

2) Class Label(클래스 레이블) - 탐지된 객체의 종류 (예: 사람, 자동차, 개 등)

3) Confidence Score(신뢰도 점수) - 모델이 객체를 특정 클래스로 예측한 확신 정도 (0~1)

4) IoU (Intersection over Union) - 예측된 바운딩 박스와 실제 정답(ground truth) 박스 간의 겹치는 정도. 값이 1에 가까울수록 정확도가 높음

5) NMS (Non-Maximum Suppression) - 중복된 바운딩 박스를 제거하여 최적의 박스만 남기는 기법

(^人^) Object Detection 주요 기법

1. 전통적인 방법 (기계 학습 기반)

과거에는 Haar Cascade, HOG (Histogram of Oriented Gradients), SIFT, SURF 등의 특징 기반 기법을 사용했지만, 딥러닝 모델에 비해 정확도와 확장성이 낮음.

2. 딥러닝 기반 방법 (Deep Learning-Based)

(1) Region Proposal 기반 (Two-Stage)

먼저 객체가 있을 가능성이 높은 영역(Region Proposal)을 찾고, 이후 CNN으로 분류하는 방식.
정확도는 높지만 속도가 느림.

대표 모델 : **R-CNN** (2014) - 후보 영역을 CNN으로 각각 분류,

Fast R-CNN (2015) - ROI Pooling으로 속도 개선,

Faster R-CNN (2016) - Region Proposal Network(RPN) 도입으로 속도 향상

(2) 단일 단계 탐지 (One-Stage)

이미지 전체를 한 번에 분석하여 객체를 탐지하는 방식.

속도가 빠르지만 정확도는 다소 낮을 수 있음.

대표 모델: **YOLO (You Only Look Once)** - 실시간 탐지 가능 (YOLOv1~v8),

SSD (Single Shot MultiBox Detector) - 다중 스케일 탐지 가능,

RetinaNet - Focal Loss를 사용하여 작은 객체 탐지 개선

(3) 최신 경량화 모델

모바일 및 임베디드 환경에서도 실행 가능하도록 최적화된 모델.

대표 모델: **EfficientDet** - EfficientNet 백본 사용,

YOLOv5~v8 - 경량화 및 성능 개선,

MobileNet-SSD - 모바일 환경에서 빠른 속도 제공

* Object Detection vs. 관련 기술

기술	주요 특징	예제
이미지 분류 (Classification)	이미지 전체에서 하나의 클래스만 예측	고양이 vs. 개 구분
객체 탐지 (Object Detection)	여러 개의 객체의 위치와 종류 탐지	사진에서 사람, 차, 동물 탐지
이미지 분할 (Segmentation)	픽셀 단위로 객체 영역을 정확히 분할	자율주행에서 도로, 보행자, 신호등 분할
객체 추적 (Object Tracking)	연속된 영상에서 특정 객체의 움직임 추적	CCTV에서 사람 추적

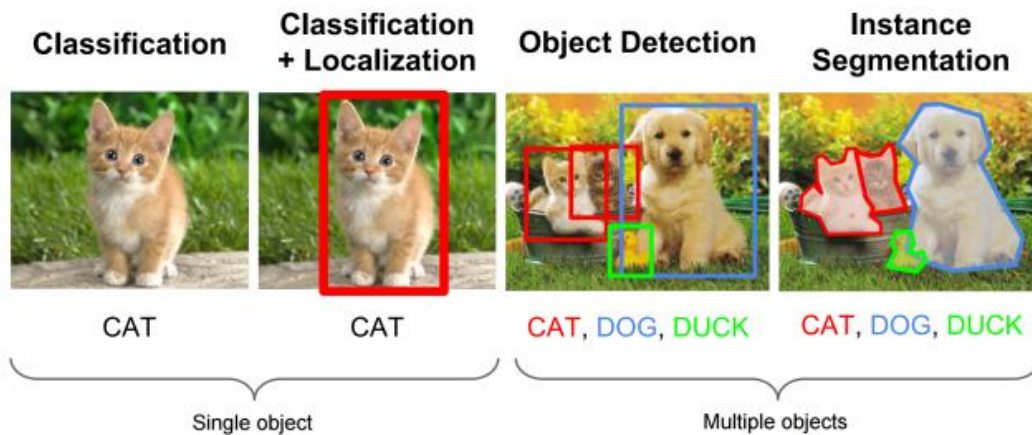
□ Object Detection 활용 분야

- 자율주행 자동차 → 도로 위 보행자, 차량 탐지
- 의료 영상 분석 → X-ray, CT에서 종양 탐지
- 스마트 보안 시스템 → CCTV를 활용한 침입 탐지
- 스포츠 분석 → 선수의 움직임 및 공의 궤적 분석
- 산업 자동화 → 결함 검사 및 품질 관리

정리하자면, **객체 탐지(Object Detection)**는 이미지 내에서 객체의 위치와 종류를 예측하는 기술이며 기존 기계 학습 방법보다 딥러닝 기반 모델(YOLO, Faster R-CNN, SSD 등)이 훨씬 강력하다. 이 때 속도 vs. 정확도 트레이드오프에 따라 YOLO(빠름) vs. Faster R-CNN(정확함) 같은 모델을 선택해야 한다.

객체 탐지는 다양한 산업에서 활용되며, 자율주행, 의료, 보안, 스포츠 등 실생활에 필수적인 기술로 자리 잡고 있다.

(^人^) 이미지에서 사물을 인식하는 방법



이미지 에서 사물을 인식하는 방법에는 크게 4가지가 있다.

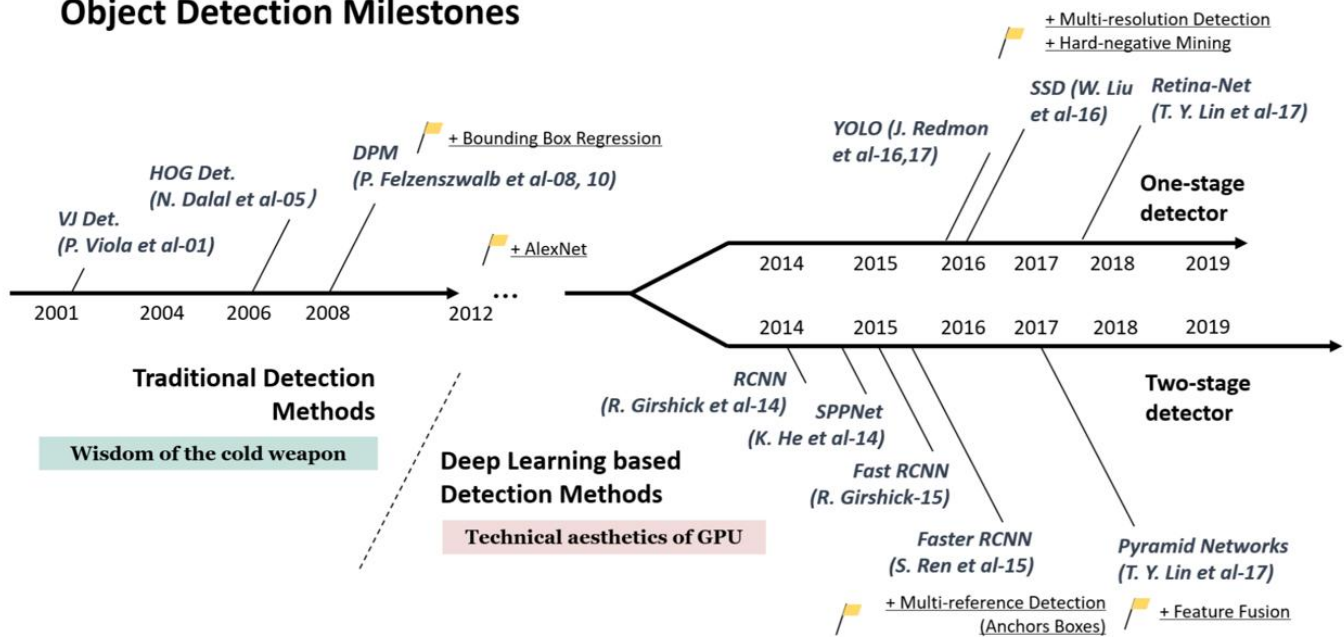
- 1) Classification이라고 불리는 전통적인 분류 문제는 하나의 이미지를 입력으로 받아 해당 이미지가 어떤 것인지 맞추는 것이다.
- 2) Localization이 추가된 방법은 해당 이미지가 어떠한 것인지 맞추고 추가로 이미지 내에서 객체가 어느 위치에 존재하는지 두 번째 사진과 같이 바운딩 박스로 표시를 해주는 것이다.
- 3) object detection 방법은 이미지 내에 다양한 사물이 존재하는 상황에, 바운딩 박스를 통해 각각의 위치와 종류를 분류하는 방법이다.
- 4) Instance segmentation은 이미지 내에 존재하는 모든 객체 각각의 인스턴스에 대해 픽셀 단위로 구분하는 방법이다.

이 중에서 object detection을 중점적으로 다뤄보겠다.

(^人^) 객체 검출의 발전 과정

객체 검출이 언제부터 어떻게 발전해 왔는지 역사를 알아보자.

Object Detection Milestones



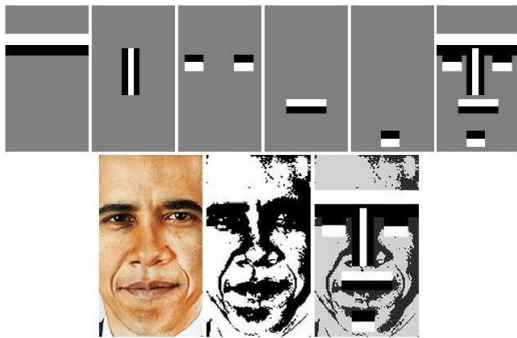
< Object Detection Milestones >

객체 인식 방법의 시초는 2001년 비올라 존스의 얼굴 감지 알고리즘으로부터 시작되었다.

2004년에 hog 특징과 svm의 결합으로 보행자 검출을 위한 hog 디텍터가 발표되었다. 그리고 2008년 hog 디텍터의 확장 버전으로 DPM이 발표되었다.

간략하게 하나씩 알아보자.

□ Viola - jones detector (haar-like feature)



객체 감지 알고리즘의 첫 번째 성공적인 구현은 2001년 비올라와 존스가 얼굴 감지 알고리즘에 사용하는 haar - like feature로 시작을 한다. 간략하게 haar 알고리즘은 사람 얼굴에서 일반적으로 나타나는 명암을 특징으로 활용한다. 얼굴에서 이마와 콧대 부분은 밝기 때문에 1의 값을, 눈가는 깊게 파여 어둡기 때문에 0 값을 주는 단순한 아이디어에서 생각해냈다.

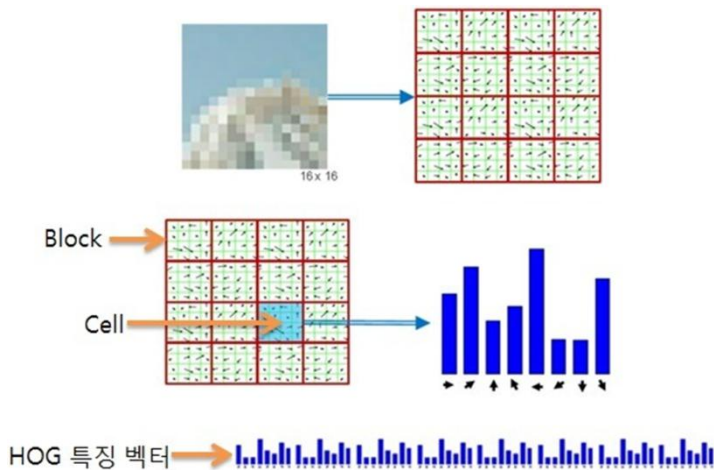
이 알고리즘은 물체 검출 성능이 높고, 특정 영역에 대한 덧셈 연산만 진행하기 때문에 고속으로 처리 가능하고 특히 얼굴 검출 실행 속도가 빠른 장점이 있다. 하지만 영역 간 밝기 차를 이용하는 만큼 광원의 방향 변화에 따른 영상 밝기 변화에 영향을 받아 물체가 회전된 경우에는 검출이 힘든 단점이 있다. 그리고 다양한 배경과 물체가 복합적으로 이루어져 있는 이미지에서는 물체 검출 시 잘못 인식할 가능성이 매우 높은 단점이 있다.

(^人^) HOG (Histograms of Oriented Gradients) detector



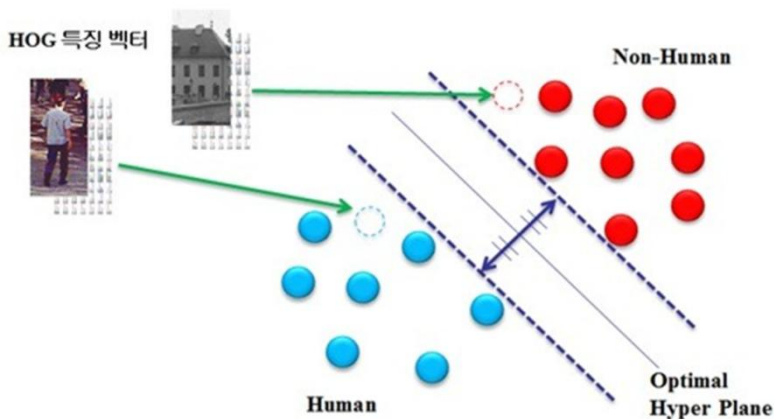
Hog 디텍터는 2005년 보행자 검출을 위해 만든 알고리즘이다. Hog는 영상의 Edge 성분을 이용해 특징을 생성하지 때문에 앞의 비올라 존스 디텍터와 달리 조명에 영향이 크지 않은 장점이 있다.

* Hog detector 원리



동작 순서를 보면 이미지를 block 단위로 나누고 block 내부의 위치한 각 셀 들에 대해 기울기에 관한 히스토그램을 만든다. 각 셀의 기울기는 이미지 인접 픽셀 간 값의 차이에 대한 gradient 값이다. 이것은 한 블록에 대한 특징 벡터가 되며, 이와 같은 과정을 이미지의 모든 블록에 적용하고 이어 붙이면 최종 1차원 hog feature 벡터를 구할 수 있다.

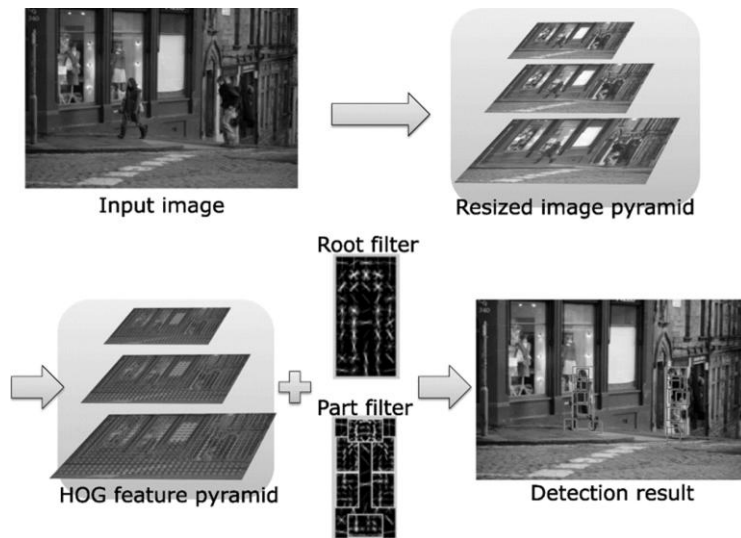
(^人^) HOG + SVM



Hog 특징 벡터가 나오게 되면 이를 서포트 벡터 머신으로 분류를 한다. 서포트 벡터 머신은 분류 과제에 사용할 수 있는 머신러닝 지도학습 모델이다. 이를 통해 사람인지 아닌지를 판단한다.

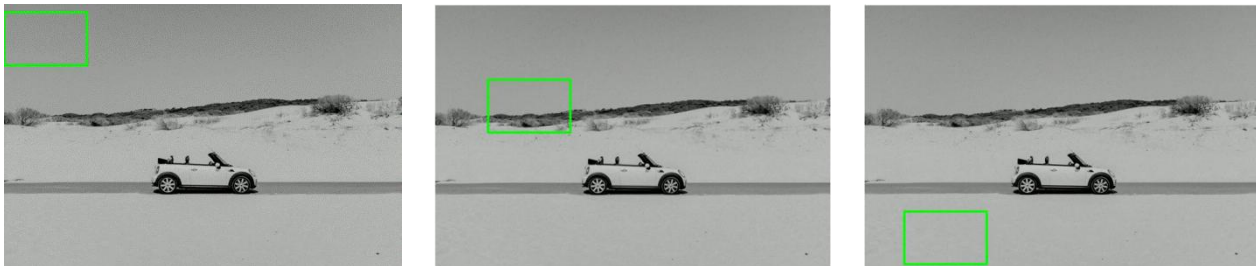
(^人^) DPM (Deformable Part Model)

위에서 말한 hog detector에 후속 연구 중 하나가 DPM이다

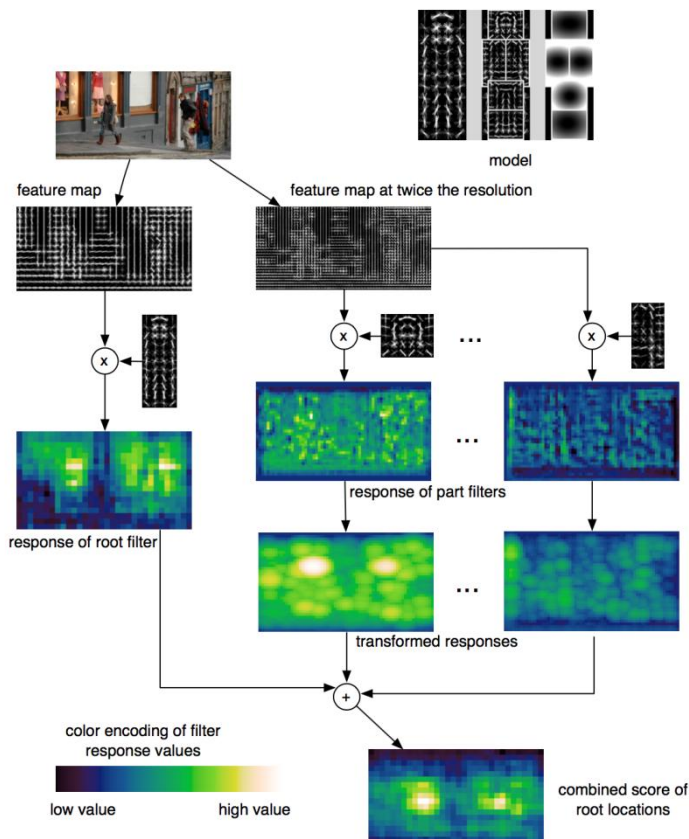


DPM은 hog 디텍터의 확장 버전으로 발표했다. DPM의 특징은 사람 객체의 각 부분 즉, 머리 몸통 다리에 대한 템플릿들을 미리 가지고 있다. 그리고 그 템플릿 들은 각각이 변형된 형태까지 가지고 있었다. 이러다 보니 당시 수준에서는 굉장히 잘 동작했다고 한다. 그리고 최초로 바운딩 박스라는 개념을 적용시켰다.

* DPM 동작 원리

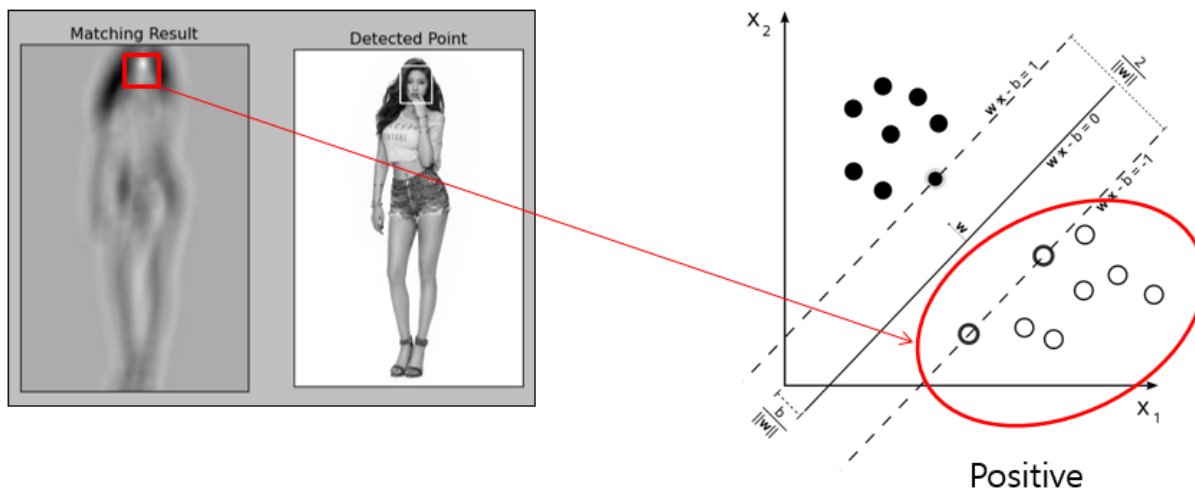


바운딩 박스를 그려 일정 크기의 픽셀마다 이동하며, 박스 영역 안의 block 마다 hog feature에 대해 히스토그램을 분석함으로 해당 block의 특징들을 선별하게 된다.



< 바운딩 박스 영역 매칭 >

그리고 미리 만들어진 인체 각 부위의 템플릿 필터들을 가지고 바운딩 박스 영역마다 매칭을 한다. 템플릿 별 일치 결과를 종합함으로써 각 객체를 분류하는 방법을 사용했다.

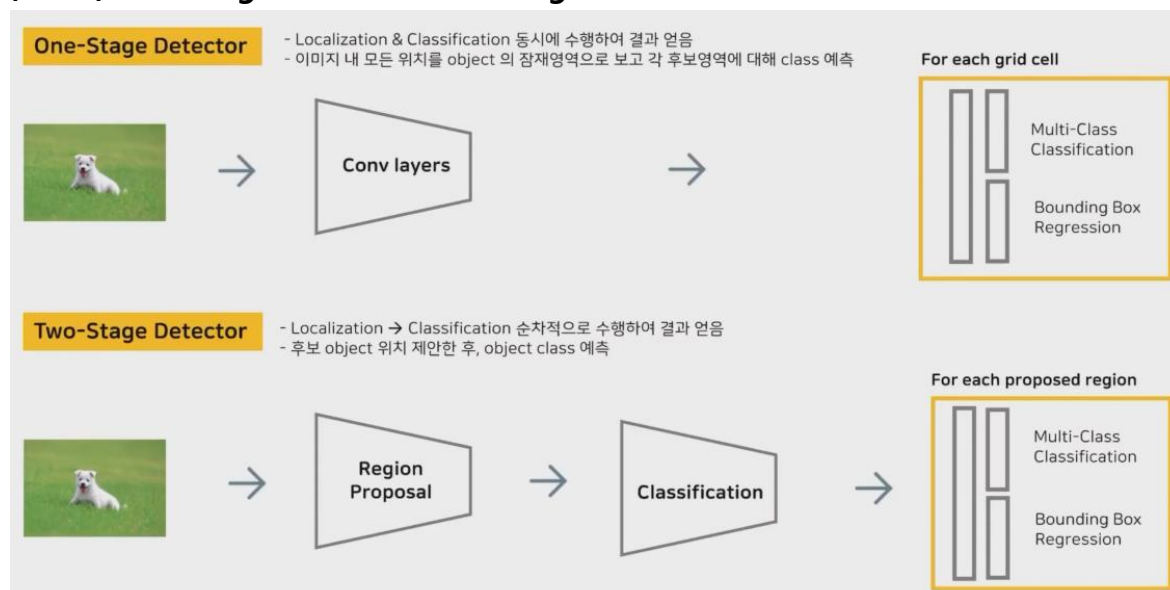


만약 템플릿이 얼굴 필터라면, 블록을 돌아다니면서 어디가 얼굴인지 svm을 통해 classification을 하게 된다. 결국 다양한 템플릿 필터를 통해 svm으로 classification 한 결과를 이용하여 최종적인 분류를 한 것이니 앙상블 기법이 적용되었다고 할 수 있다.

하지만 DPM에는 문제점이 있다. 슬라이딩 윈도우를 통해 이미지의 모든 부분을 일일이 계산하면 굉장히 많은 바운딩 박스가 형성될 것이고 각각의 바운딩 박스마다 복잡한 classification 과정을 거치게 된다. 이렇게 되면 background를 검출하는 잉여시간이 너무 많아진다. 이러한 문제들로 인해 object detection 영역에서 힘을 못쓰다가 CNN분야인 AlexNet의 등장으로 객체인식이 획기적인 발전을 하게 되는데 그 모델이 R-CNN이다.

딥러닝 이후로는 1stage detector와 2 stage detector 2가지 방법으로 나뉘게 된다.

(^人^) One stage detector, Two stage detector



Object Detection은 물체를 식별하는 Classification 문제와, 물체의 위치를 찾는 Localization 문제를 합한

것이다. 다시 말해 1-stage Detector는 이 두 가지 task를 동시에 행하는 방법이고, 2-stage Detector는 이 두 문제를 순차적으로 행하는 방법이다.

이번에는 2 stage detector에 대해 알아 보자.

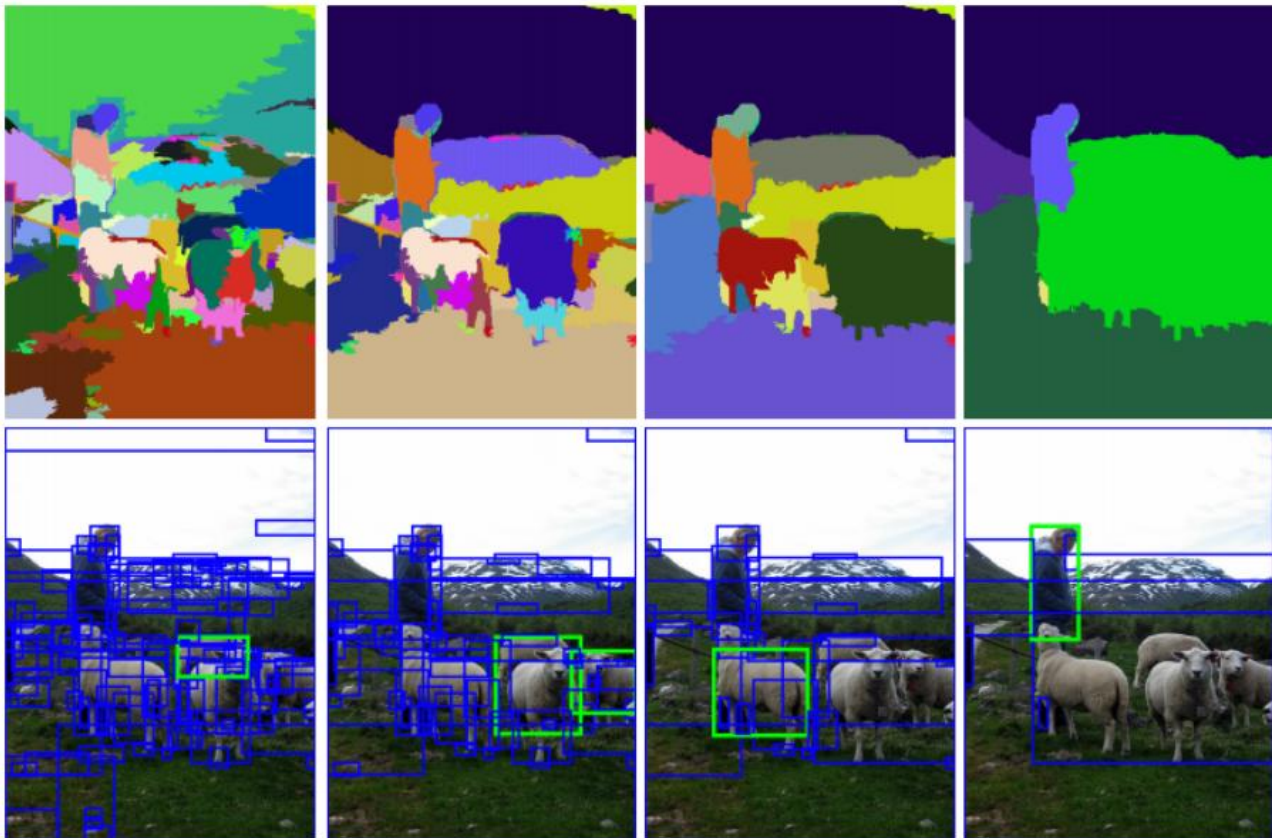
편의상 Object detection 분야에 딥러닝을 최초로 적용시킨 모델인 R-CNN부터 알아보자.

(^人^) R - CNN (Regions with CNN features)

R-CNN은 Region With Convolution Neuron Networks features의 약자로 즉 설정한 region을 CNN의 feature(입력값)로 활용하여 Object detection을 수행하는 신경망이라는 의미이다.

설명에 앞서 selective search에 대해 알아야 하기에 해당 설명부터 하고 넘어가자.

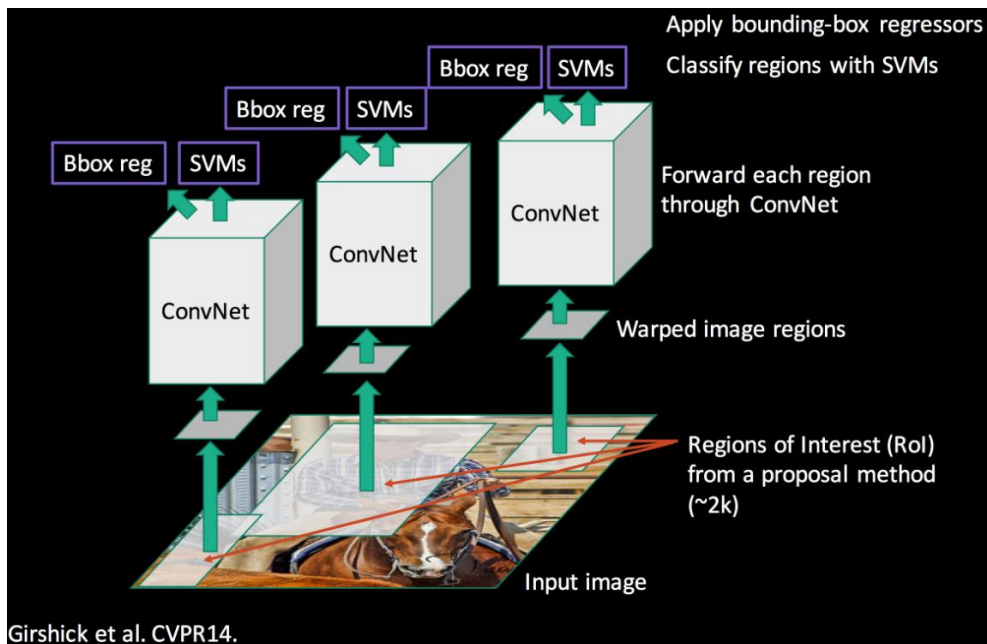
□ Selective Search



출처: selective search paper with code

selective search 란 객체의 위치를 찾는 방법이다. 기존의 슬라이딩 윈도우를 통한 이미지 모든 영역에 대한 search 방식의 비효율성으로 이제는 object가 있을 법한 영역만 찾는 selective search 방법이 나오게 되었다. 과정을 보면 입력 이미지에 대해 segmentation을 실시해서 이를 기반으로 후보 영역을 찾는다. 초기에는 후보영역이 많이 만들어지는데 유사한 영역끼리 적절하게 통합해 나가면서 바운딩 박스의 수는 줄어들게 된다. 유사한 후보영역을 찾는 기준은 색깔과 주변 픽셀 값들의 변화량 그리고 바운딩 박스의 크기 차이로 정해 진다.

(^人^) R - CNN 동작원리



동작 순서를 보자면

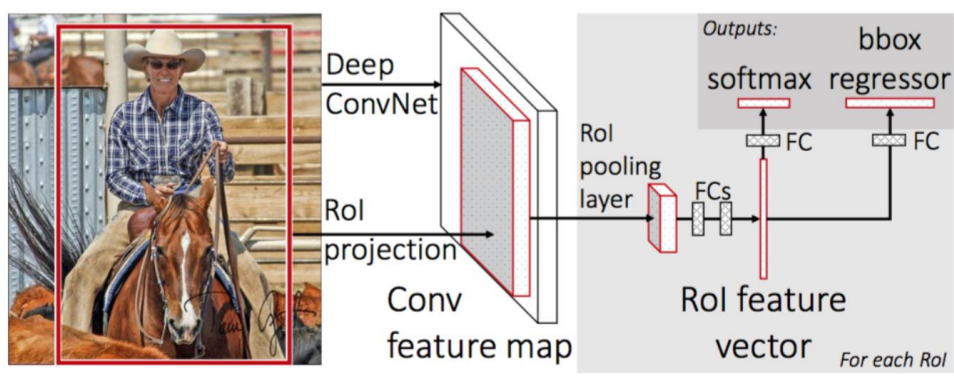
1. 인풋 이미지에 대해 selective search 알고리즘을 적용해 물체가 존재할 법한 위치 2000개 가량을 찾는다. (여기서 물체가 존재할만한 위치를 관심 영역, 영어로 regions of interest 줄여서 ROI라고 하자)
2. 이 2000개의 ROI를 동일한 크기로 warping 한 후 동일한 크기의 이미지로 변경한다
3. Warped image를 CNN에 넣어 feature를 추출한다. 찾은 feature 들에 대해 이진(binary) svm에 넣어 class 분류 결과를 얻는다. 그리고 동시에 이 feature 들어 대해 regression을 적용하여 바운딩 박스의 위치를 예측한다.

최초의 R-CNN은 좋은 성능을 냈지만 몇 가지 한계점이 존재했다.

- 입력 이미지에 대한 CPU 기반의 selective search 과정 자체가 시간이 많이 걸림
- 찾아낸 2000개의 영역을 각각 CNN을 통해 특징 추출을 하기에 연산량이 많다
- CNN과, Classification, 바운딩 박스 Regression 이 세 가지를 따로 학습해 함으로 학습이 느렸고 테스트에 시간이 오래 걸림

이러한 문제점을 해결하기 위해 Fast R-CNN이 등장했다.

(^人^) Fast R - CNN



Fast R-CNN은 selective search에서 얻어진 roi들을 각각 cnn에 넣는 방식이 아닌 이미지 통째로 cnn에

넣는 방식이다. 이 방식을 이용해 R-CNN에서 CNN 연산을 2000번 하던 것을 Fast R-CNN는 한 번으로 줄였다.

동작 순서를 보면

1. Fast R-CNN 은 R-CNN과 같이 selective search를 통해 ROI를 찾고 동시에 전체 이미지를 CNN에 통과시켜 feature map을 추출한다.

2. 이제 selective search로 찾았었던 ROI를 feature map크기에 맞춰 투영시켜 피쳐 맵 상에서의 ROI를 찾는다.

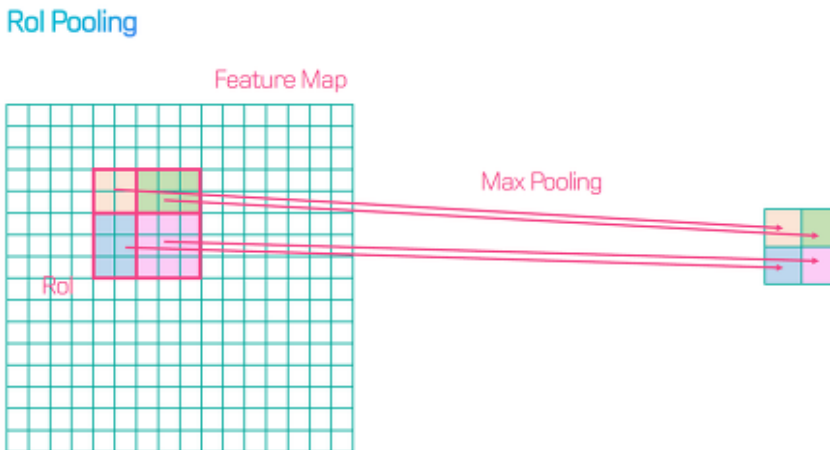
3. 투영시킨 ROI에 대해 RoI Pooling을 진행하여 고정된 크기의 feature 벡터를 얻는다. (여기서 RoI Pooling 은 ROI에 해당하는 부분만 max pooling을 진행하여 피쳐 맵으로부터 저 차원 벡터로 축소하는 단계다)

4. 이 feature벡터 들은 fully connected layer을 통과한 뒤 하나는 softmax 함수를 통과하여 물체를 분류하고, 하나는 바운딩 박스 regressor를 통해 바운딩 박스 위치를 조절한다.

이전에는 2000번의 cnn 연산을 하였지만 한 번의 연산으로 속도가 빨라졌다. 그리고 동일 데이터가 동시에 softmax와 바운딩 박스 regressor로 들어가 연산 과정을 공유하기에 end-to-end 즉, 한 번에 학습이 가능하다.

하지만 ROI를 생성하는 selective search 알고리즘은 여전히 CPU 상에서 동작하고 이로 인해 병목현상이 발생하게 된다. 이러한 문제를 해결하기 위해 Faster R-CNN 이 등장했다.

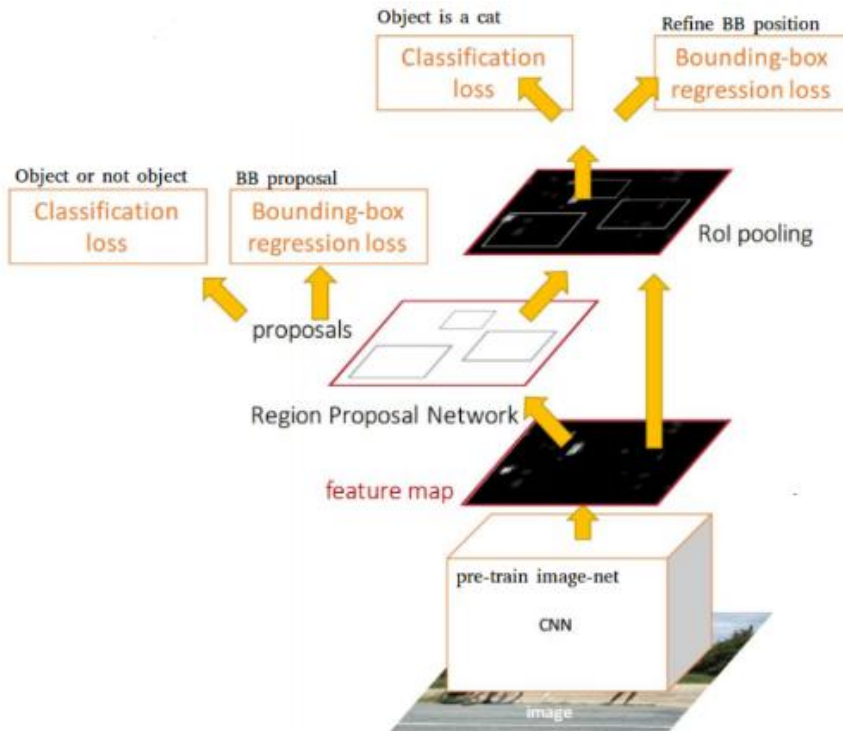
참고 : ROI - Pooling



ROI-Pooling은 위 그림과 같이 피쳐 맵에 사전에 정해진 그리드 셀을 가로 세로 절반으로 나눠 Max Pooling을 해줘 고정된 크기의 region을 만드는 과정이다. 이를 하는 이유는 기본적으로 분류를 위해서 Fully Connected Layer을 이용하는데 FC layer는 고정된 input 크기를 요구하기 때문에 크기가 제 각각인 region을 고정된 크기로 바꿔줄 필요가 있기 때문이다.

하지만 정확하게 셀을 절반으로 나눠지지 않을 때는 이미지가 변형될 수 있다. (이를 방지하기 위한 것이 ROI align)

(^人^) Faster R - CNN



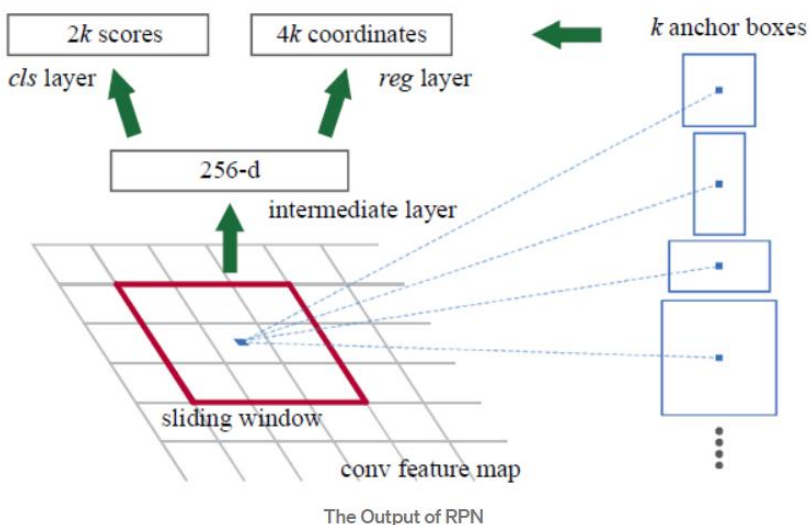
Faster R-CNN 은 기존의 cpu계산으로 시간이 걸렸던 selective search 부분을 RPN (Region Proposal Network)으로 대체하여 속도 문제를 해결하였다.

RPN은 GPU에서 수행되는데 이 덕분에 GPU를 통해 ROI 연산이 가능해졌으며, ROI 연산도 학습을 시켜 정확도를 높였다. 모델을 보면 RPN과 Fast R-CNN 모델이 합쳐졌다고 볼 수 있다.

RPN에서 region proposal를 추출하고 이를 Fast R-CNN 네트워크에 전달하여 객체의 class와 위치를 예측한다. 이를 통해 모델의 전체 과정이 GPU 상에서 동작하여 병목 현상이 발생하지 않으며, 출력부터 입력까지 backpropagation을 통해서 gradient를 계산할 수 있다는 장점이 있다. 즉, end-to-end로 네트워크를 학습시키는 것이 가능해졌다는 이야기다.

Fast R-CNN의 앞 부분인 selective search 부분만 RPN으로 대체되었기에 RPN만 추가로 알아보자.

□ RPN (Region Proposal Network)



RPN은 region proposal network의 약자로 기존 selective search를 대체한 방식이다. RPN의 입력 값으로는 원본 이미지에서 cnn을 거쳐 나온 feature map이 들어간다. 그 feature map위에 sliding window

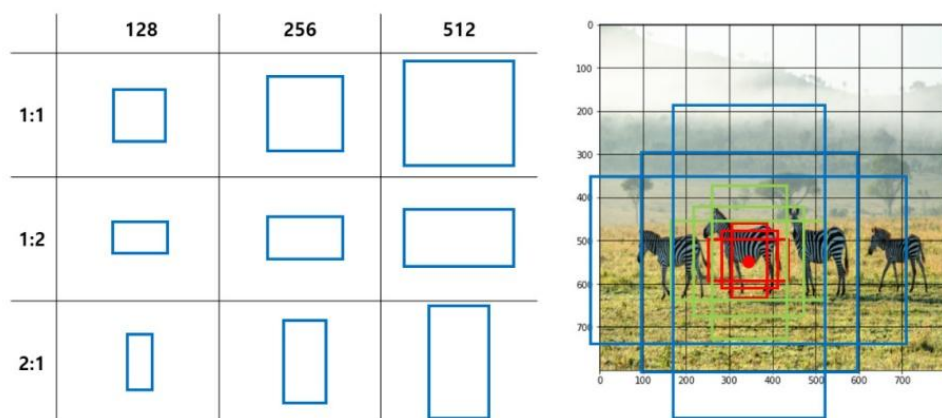
를 사용하며 임의의 k개의 앵커 박스를 사용한다. (앵커 박스는 하단에 설명 추가함)

이렇게 만들어진 앵커 박스들에 대하여 classification 계층과 regression 계층으로 들어가게 된다. classification 계층에서는 박스 내부 객체 존재 유무에 대해 1과 0으로 출력한다. regression 계층에서는 박스 중앙의 x, y 좌표, 높이, 너비 이렇게 4가지 정보를 출력한다.

이 RPN단계에서 분류와 회귀를 하는 이유는 결국 학습을 위함이다.

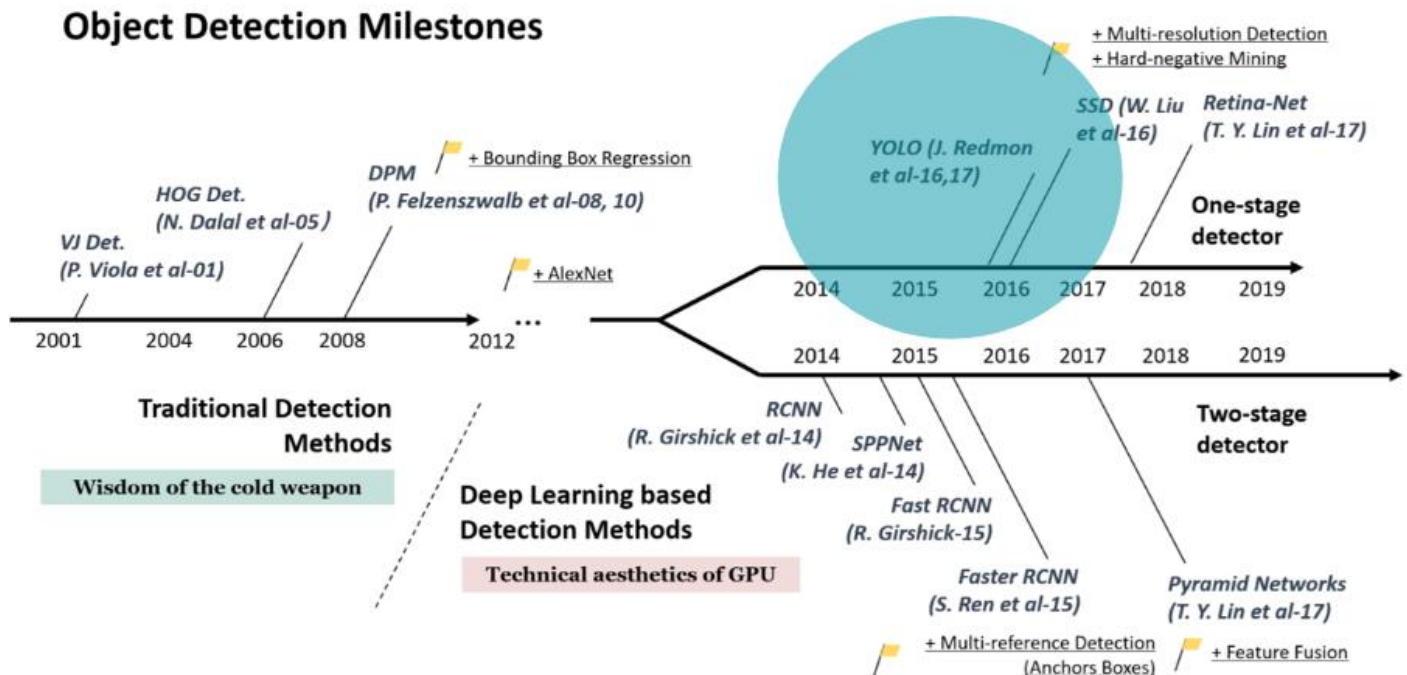
여기까지가 RPN의 과정이며 이 결과 값이 Fast R-CNN과 동일한 나머지 과정을 거치게 된다. 이로써 CPU에서 하던 selective search과정이 GPU상에서 RPN으로 돌아가게 됨으로 속도가 개선되었다.

(^人^) Anchor box



앵커 박스란 고정된 크기의 바운딩 박스를 사용할 경우 다양한 크기의 객체를 포착하지 못할 수 있다는 문제가 있어 만들어진 서로 다른 크기의 바운딩 박스들을 말한다.

^^^ YOLO (you only look once) ^^^



YOLO (You Only Look Once)란?

: 실시간 객체 탐지를 위한 딥러닝 모델로, 한 번의 신경망 순전파(Forward Pass)로 이미지를 분석하여 객체를 탐지하는 방식이다.

YOLO의 주요 버전 발전

- YOLOv1 (2016): 단순한 구조이지만 정확도와 속도 균형이 필요함.
- YOLOv2 (2017, YOLO9000): 속도 향상, 작은 객체 탐지 성능 개선.
- YOLOv3 (2018): 다중 스케일 객체 탐지 적용, 성능 향상.
- YOLOv4 (2020): CSPDarknet 백본 적용, 실시간 성능 극대화.
- YOLOv5 (2020): PyTorch 기반, 성능 최적화 및 가벼운 모델.
- YOLOv6, YOLOv7 (2022): 경량화, 속도/정확도 향상.
- YOLOv8 (2023): 최신 모델로, 오브젝트 추적 및 분할 지원.
- ...

--- YOLOv8 버전 종류 ---

Ultralytics에서 새롭게 선보인 YOLOv8은 용도 및 모델 크기에 따라 여러 가지 버전(모델 사이즈)을 제공한다. 일반적으로 사용하는 모델은 다음과 같은 네이밍 규칙을 갖는다.

- YOLOv8n (nano)
- YOLOv8s (small)
- YOLOv8m (medium)
- YOLOv8l (large)
- YOLOv8x (xlarge)

이 모델들은 크기와 성능이 트레이드오프 관계에 있다.
예컨대, n(nano)는 연산량이 가장 적고 속도가 빠르지만 정확도가 낮은 편이며, x(xlarge)는 연산량이 많아 느린 대신 정확도가 높은 편이다.

YOLO의 주요 활용 분야

- 자율주행 차량 - 도로 객체 탐지
- CCTV 및 보안 - 침입자 감지
- 의료 영상 분석 - X-ray, CT에서 병변 탐지
- 스포츠 분석 - 선수 움직임 추적
- 드론 및 로봇 비전 - 실시간 객체 인식

“한 마디로 YOLO는 속도가 중요한 실시간 객체 탐지에서 강력한 성능을 발휘하는 대표적인 모델이다!” 기존의 2 stage detection 방식의 느린 속도 때문에 이 둘을 한 번에 처리하는 1 stage detection 방식이 나오게 되며, 그 시초 모델이 YOLO이다. YOLO는 classification 과 Localization을 동시에 하여 실시간으로 객체의 검출이 가능하다!

Yolo가 유명해진 이유는 높은 성능은 아니더라도 실시간으로 object detection이 가능하고 기존의 Faster R-CNN 보다 6배 빠른 성능을 보였기 때문이다. 하지만 R-CNN계열 모델들에 비해 낮은 정확도를 가지고 있으며 특히 작은 객체에 대해 정확도가 아쉬울 정도로 낮은 편이다.

***** YOLO는 객체 탐지(Object Detection) 모델 중 하나로, 단일 신경망으로 전체 이미지를 한 번에 처리하는 것이 특징이다. *****

Yolo는 Single neural network를 사용한다.

순서 : Resize image -> Run convolutional network -> Non-max suppression



@ YOLO 처리 순서 :

- 1) Resize image : 입력 이미지를 YOLO 모델이 처리할 수 있도록 고정된 크기(예: 416x416, 640x640 등)로 리사이즈한다. 이 단계는 모델 입력에 맞게 모든 이미지를 정규화시키기 위함이다.
- 2) Run convolutional network (CNN) : 리사이즈된 이미지를 단일 CNN에 입력하면, 이미지 전체를 한 번에 훑으면서 바운딩 박스 좌표(x, y, w, h), 클래스 확률(class probability), confidence score를 예측한다. 이 과정을 통해 각 그리드 셀이 여러 개의 박스를 예측하게 된다.
- 3) Non-Max Suppression (NMS) : 같은 객체에 대해 여러 박스가 예측될 수 있기 때문에, 신뢰도(confidence score)가 높은 박스를 남기고, 겹치는 박스를 제거하는 비최대 억제(NMS) 과정을 적용한다. 이로써 최종적으로 가장 적절한 바운딩 박스만 남게 된다.

@ 특징 :

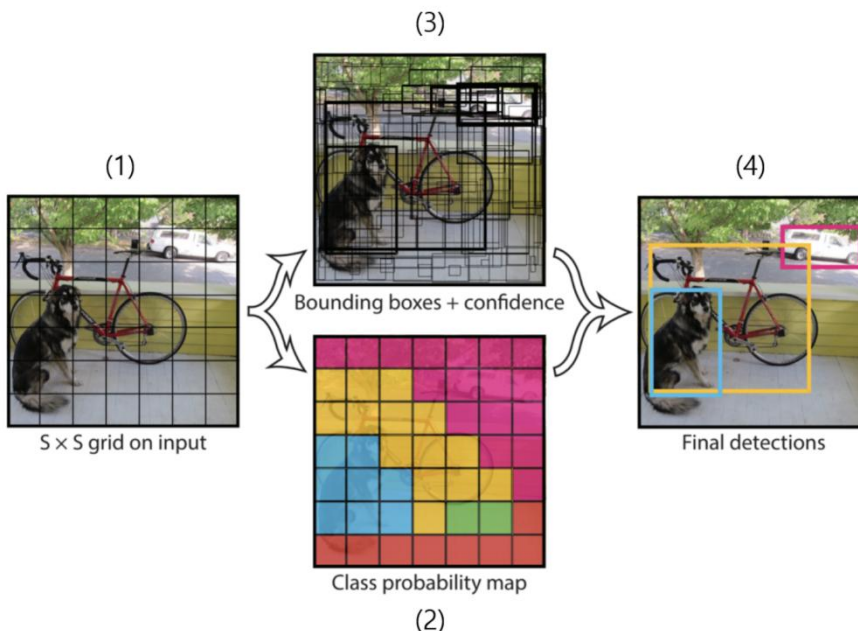
- 속도 빠름: 실시간 객체 탐지에 적합
- 단일 패스 구조: 한 번의 forward pass로 모든 객체 예측
- End-to-End 학습 가능: 이미지 → 박스 + 클래스까지 한꺼번에 학습

YOLO 작동 원리 간단 요약

: 이미지를 입력 받아 $S \times S$ 그리드로 나눔.

- ➔ 각 그리드 셀에서 바운딩 박스, 신뢰도, 클래스 예측.
- ➔ NMS(Non-Maximum Suppression) 적용하여 중복 박스 제거.
- ➔ 최종 객체 감지 결과 출력.

□ YOLO (you only look once) 동작과정



먼저 사진이 입력되면

(1) 입력 이미지에 $S \times S$ 그리드 적용 : 입력 이미지를 $S \times S$ 크기의 격자로 나눔. 각 격자 셀은 해당 셀 안에 중심이 포함된 객체를 탐지할 책임을 가짐.

(2) Class Probability Map (클래스 확률 맵) : 각 그리드 셀은 해당 영역에 특정 객체(클래스)가 있을 확률을 예측. 예: "이 셀에는 개가 있을 확률 80%", "자전거일 확률 10%" 등

(3) Bounding Boxes + Confidence : 각 그리드 셀은 여러 개의 바운딩 박스 좌표(위치, 크기)와 신뢰도 (Confidence) 점수를 출력함. 신뢰도는 "이 박스가 실제 객체를 포함할 가능성"을 나타냄

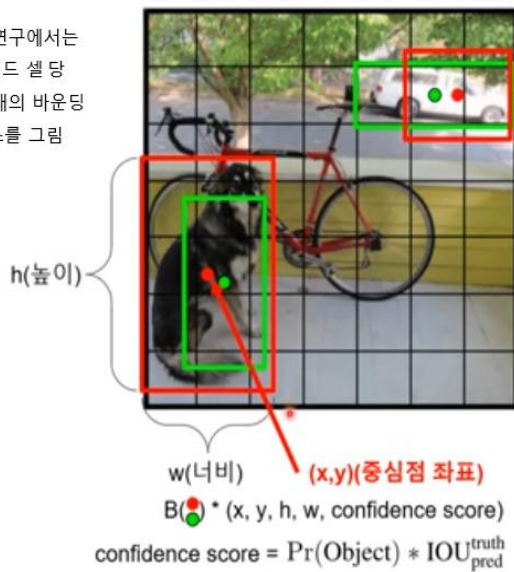
(4) Final Detections (최종 탐지 결과) : 클래스 확률 $\times$ 바운딩 박스 신뢰도를 곱해 최종 점수 계산한다. NMS(Non-Maximum Suppression) 등을 통해 중복되는 박스를 제거하고 가장 신뢰 높은 탐지 결과만 출력됨.

이처럼 YOLO는 이미지를 격자로 나눔 뒤 각 셀에서 클래스 확률과 바운딩 박스를 예측하고, 최종적으로 신뢰도 높은 탐지 결과만 추려낸다.

"Detection as a regression problem"

YOLO는 객체 탐지를 회귀 문제로 접근하여, 위치(x, y, w, h)와 클래스 확률을 한 번에 예측하는 구조를 가지며, 이것이 바로 "Unified Detection"이다.

이 연구에서는
그리드 셀 당
두 개의 바운딩
박스를 그림



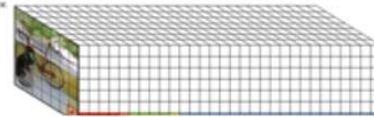
개, 소, 자동차, 자전거, 말, 사자
conditional class probability = $\text{Pr}(\text{Class}_i | \text{Object})$

자동차	자동차	자동차	자동차	자동차	자동차	자동차
자전거	자전거	자전거	자동차	자동차	자동차	자동차
자전거	자전거	자전거	자동차	자동차	자동차	자동차
개	개	자전거	자전거	자동차	자동차	자동차
개	개	개	자전거	자전거	자동차	자동차
개	개	개	자전거			

Figure 2: The Model. Our system models detection as a regression problem. It divides the image into an $S \times S$ grid and for each grid cell predicts B bounding boxes, confidence for those boxes, and C class probabilities. These predictions are encoded as an $S \times S \times (B * 5 + C)$ tensor.

For evaluating YOLO on PASCAL VOC, we use $S = 7$, $B = 2$. PASCAL VOC has 20 labelled classes so $C = 20$. Our final prediction is a $7 \times 7 \times 30$ tensor.

$7 \times 7 \times (2 * 5 + 20) = 7 \times 7 \times 30 = 1,470$



이 논문에서 바운딩박스 2개, 예측할 값 5개, 클래스는 20개 (셀 마다 예측할 값은 30개임). 전체 이미지에서 1470개가 예측될 값임

confidence score

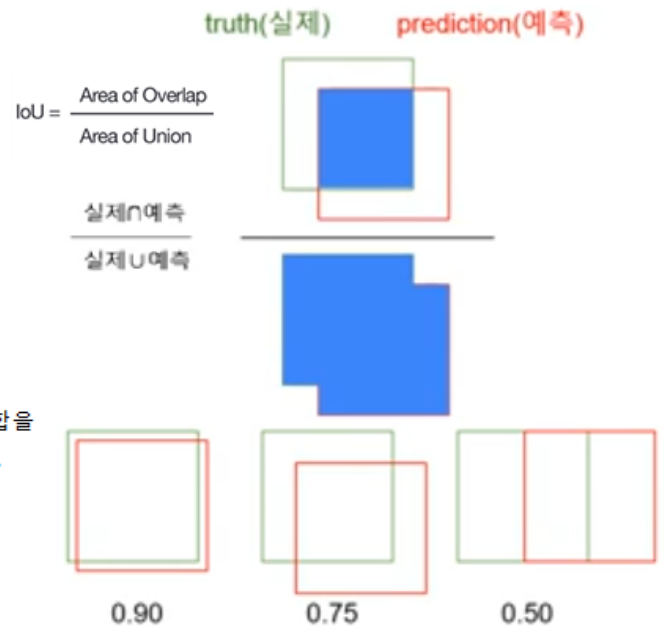
$$\text{Pr}(\text{Object}) * \text{IOU}_{\text{truth_pred}}$$

(객체가 확실히 없으면 0) * IOU = 0
(객체가 확실히 있으면 1) * IOU = IOU

IOU (intersection Over Union)

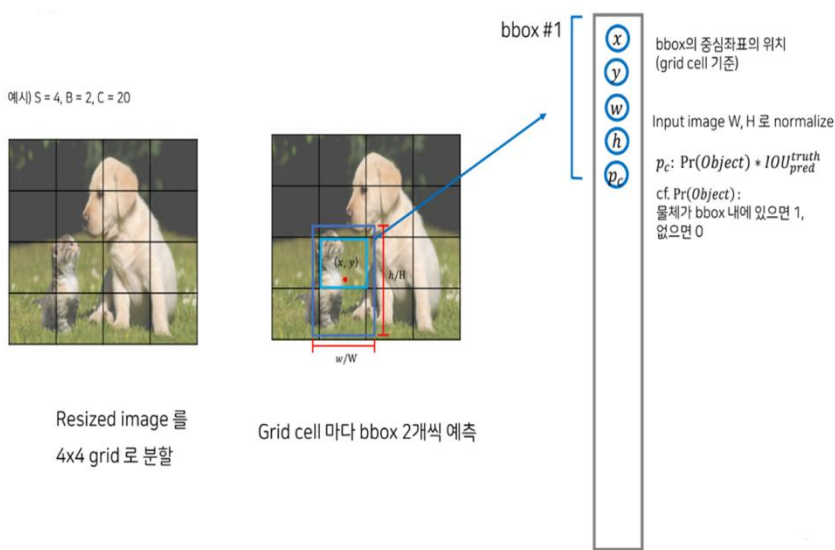
IOU는 바운딩 박스의 위치 정확도에 대한 평가 도구다.

예측한 바운딩 박스와 실제 물체의 바운딩 박스 두 개의 교집합을 총 너비로 나눈 값이며, 수치가 클수록 정확도가 높다 볼 수 있다.



동작과정에 대해 조금 더 구체적으로 예시를 들어 알아보자.

□ YOLO 동작 과정 예시



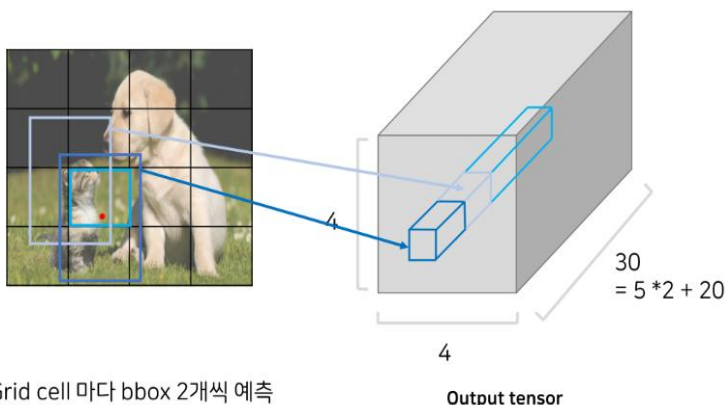
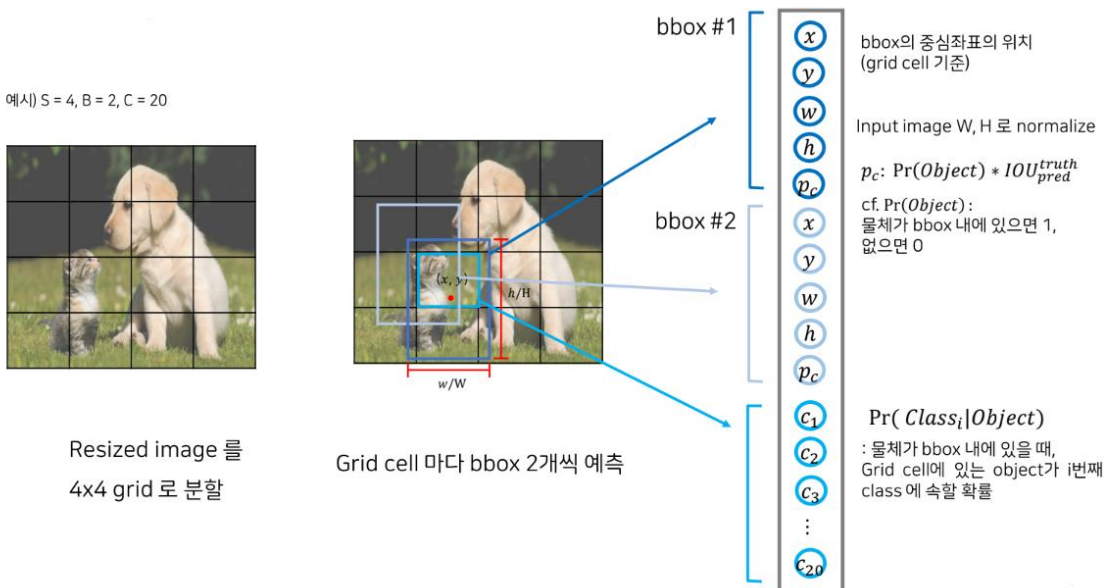
이미지를 4x4 그리드로 나눈다 가정하고(실제 논문에서는 7x7), 각 그리드 셀마다 예측하는 바운딩 박스는 2개이며 전체 고려하는 class 개수는 20개로 가정을 해보자.

첫 번째 예측된 바운딩 박스는 파란색이며 이때 예측된 바운딩 박스의 정 중앙 좌표인 x, y 좌표 그리고 바운딩 박스의 너비와 높이를 전체 이미지로 나눠 노멀라이즈 한 w 와 h (0~1 사이의 값을 가짐) 그리고 박스 내부에 물체가 존재할 확률

confidence score인 p_c 까지 총 5가지 아웃풋이 나오게 된다.

여기서 confidence score인 p_c 는 p_r 에 IOU를 곱해준 값이다. p_r 은 해당 바운딩 박스 내부에 물체가 있으면 1 없으면 0 값이며, IOU는 간단하게 설명하고 넘어가겠다.

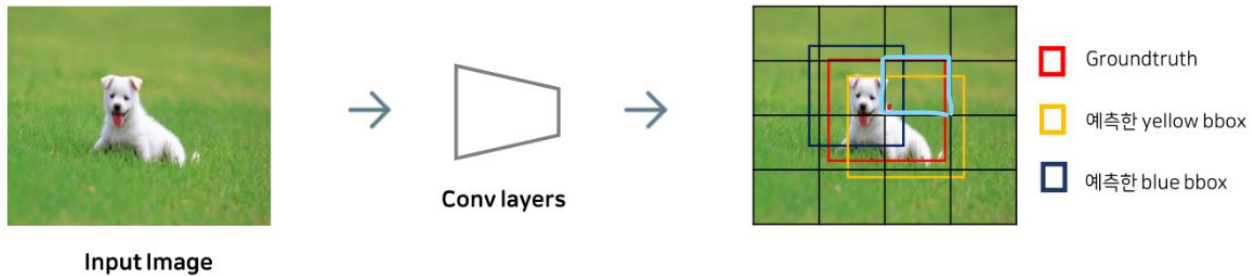
□ YOLO 동작 과정 예시 (이어서 설명)



위에서 설명했던 과정을 한 번 더 하여 두 번째 바운딩 박스에 대해 예측한 5가지 값에 대해 1차원 텐서로 넣어준다. 마지막으로 하늘색 박스를 보면 그리드 셀에 있는 오브젝트가 어떤 클래스 인지 확률이 들어가게 된다. 이로써 하나의 그리드 셀에 대한 아웃풋이 나오게 되며, 이 과정을 모든 그리드 셀 (16개)에 대해 적용시켜 준다.

□ Training Stage

위에서는 YOLO의 inference 동작 원리에 초점이 맞춰져서 설명을 했다면, 이제는 학습이 어떻게 이루어지는지 한번 알아보자. 위에서 설명한 inference 과정에서는 바운딩 박스가 여러 개 등장했지만 학습과정에 참여하는 바운딩 박스는 딱 하나! 그 하나의 바운딩 박스를 선정하는 과정을 보자.



특정 object에 대해 responsible 한 cell을 찾아야 하는데, 이는 Ground truth 박스의 중심 좌표가 위치하는 셀로 할당하게 된다. 위 그림에서 보면 하늘색 그리드 셀이 강아지를 찾는데 responsible한 셀이 되는 것이다. (정답 강아지의 Ground truth 박스의 중앙이(빨간 점) 위치하기에) 이때, 하늘색 셀의 영역에서 두 개의 노란색과 남색 박스를 예측했다고 가정하자. (남색 박스와 노란색 박스는 conv layers를 거쳐 나온 예측된 박스를 의미한다.) 여기서 하나의 바운딩 박스를 고르는 기준은 IOU 값이다. Ground truth와 겹치는 IOU 값이 더 높은 노란색 바운딩 박스만이 학습에 참여하는 것이다.

이어서 예측한 노란 박스와 GT박스와 loss function을 어떻게 계산하는지 설명하겠다.

□ Loss Function (손실 함수)

training 단계에서 사용하는 loss는 (SSE) Sum of Squared Error를 사용한다.

손실 함수는 인공 신경망 모델을 학습할 때 사용된다. 예측값과 정답 레이블 간의 오차를 측정하기 위해 사용되며, 이 오차를 최소화함으로써 인공 신경망 모델의 성능을 향상시킨다.

$\mathbb{1}_{ij}^{obj}$ i번째 그리드셀의 j번째 바운딩박스가 객체하나를 책임지고 있는지 여부 (1, 0)

Regression loss

i: 그리드셀
j: 바운딩박스

localization의 비중을 늘리기 위한 값(논문에서는 5)

$$\lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right]$$

바운딩박스의 크기에 따른 오차 차별을 없애기 위해 루트를 씌움

$$+ \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right]$$

Confidence loss

객체가 없는 부분의 비중을 줄이기 위한 값(논문에서는 0.5)

$$+ \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} (C_i - \hat{C}_i)^2$$

$$+ \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{noobj} (C_i - \hat{C}_i)^2$$

Classification loss

$$+ \sum_{i=0}^{S^2} \mathbb{1}_i^{obj} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2$$

S { }

yolo의 loss는 b-box regression loss, confidence loss, classification loss 이렇게 3가지가 있다.

- 파란 박스에 대해 간략하게 설명하자면 regression loss인데 모든 grid cell에서 예측한 B개의 바운딩 박스의 좌표(x, y, width, height)와 GT box 좌표의 오차를 구하는 공식이다.
 - 빨간 박스는 confidence loss인데 모든 grid cell에서 예측한 B개의 클래스에 속할 확률 값과 GT 값의 오차를 구하는 공식이다
 - 초록 박스는 classification loss인데 cell 내에 클래스가 존재할 확률을 구하는 공식이다.
- 여기서 공통적으로 i는 cell의 index, j는 bounding box predictor index (여기서는 2개 존재)이다.

먼저 regression loss부터 알아보자

~ B-box regression Loss

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right]$$

<bounding box regression loss>

S^2 : 시그마 윗부분에 S^2 에서 S는 (s, s) 즉 feature map의 grid 수이다.

B: 두 번째 시그마 윗부분에서 B는 Anchor Box의 수이다.

$\mathbb{1}_{ij}^{\text{obj}}$ ← 이건 obj라고 되어 있으니 object와 관련이 있는 것이라고 생각된다. 이게 논문에서 말하는 responsible이나 아니냐를 나타내는 것이다. i 번째 cell 안에 2개의 detector 중에서 j 번째 detector가 responsible이면 1이고, not responsible이면 0이다. 다시 말해 object의 중심이 속하는 i 번째 cell 이면서 해당 cell의 예측된 bbox 2개 중 object와 가장 높은 IOU를 갖는 1개의 bbox가 responsible이 된다. 이 것은 모든 grid cell에 대해 loss를 연산하지 않고 object에 해당하는 셀, b-box에 대해서만 연산을 함으로 효율적으로 연산하도록 한 것이다.

$\left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right]$ 대괄호 안의 식은 예측한 바운딩 박스의 중심 좌표 $\hat{x}_i, \hat{y}_i$ 와 ground truth의 중심 좌표 x_i, y_i 의 오차 제곱을 나타내는 식이다.

$\left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right]$ width, height 도 마찬가지로 오차 제곱으로 나타나 있다. 하지만 루트가 씌여져 있는데 이는 object의 절대적인 값이 큰 경우에 절대적인 오류도 커지는 곡을 막기 위해 나타낸 것이다.

λ_{coord} ← 가장 앞의 람다는 스케일링을 적용한 것인데, 이는 뒤에 나오는 다른 두 개의 로스들과 일종의 균형을 맞춰주기 위한 벨런싱 파라미터라고 볼 수 있다. (보통 5로 설정된다)

~ Object Confidence Loss

$$\begin{aligned}
 & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left(C_i - \hat{C}_i \right)^2 \\
 & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} \left(C_i - \hat{C}_i \right)^2
 \end{aligned}
 \quad < \text{Confidence Loss} >$$

Object Confidence Loss는 예측한 object의 loss를 나타낸다.

C_i : 모델에서 구한 object일 확률 값과 IOU를 곱해 얻게 된다.

$\hat{C}_i$: Ground truth confidence score 로써 값은 1이다.

$\mathbb{1}_{ij}^{\text{noobj}}$ ◀ 여기서 주목해야 하는 것은 object가 없어야 하는 확률에 대해서도 b-box의 confidence loss를 구해서 포함 함 시킨다는 점이다.

λ_{noobj} ◀ noobj loss 에는 람다 가중치를 곱해준다 보통 값은 0.5이다. object 객체가 탐지되지 않은 영역에 대한 loss를 0.5 곱해줘 낮춰 주는 이유는 클래스 불균형 문제를 해소하기 위함이다. 대부분의 b box는 오브젝트를 포함하고 없을 것이기 때문에 배경에 대해서도 가중을 똑같이 1로 주게 된다면 모델은 배경 학습에 더 집중할 수 있기 때문이다.

~ Classification Loss

$$+ \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} \left(p_i(c) - \hat{p}_i(c) \right)^2$$

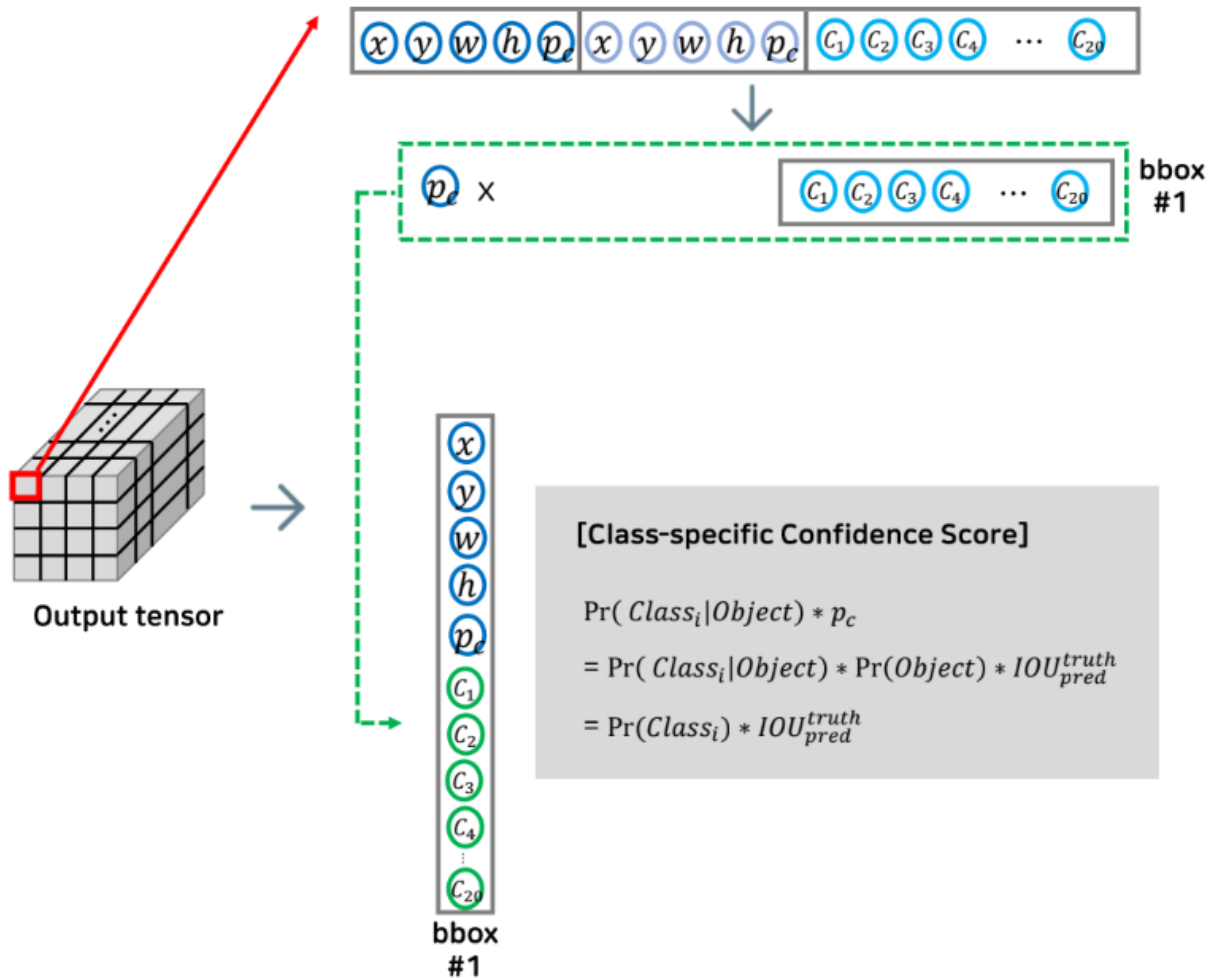
< Classification loss >

classification loss는 말 그대로 클래스별 확률 값에 대한 오차 제곱으로 구해 주게 된다.

$p_i(c)$: 해당 cell i 내부에 존재하는 object가 c class 일 확률이다.

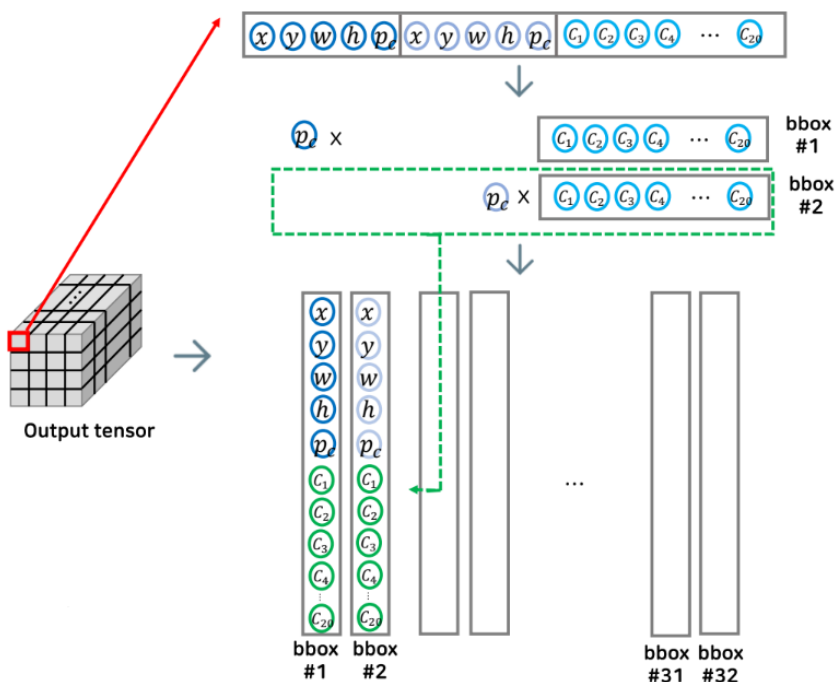
□ Inference stage

이제 어떤 사물인지 추론하는 단계에 대해 알아보자.



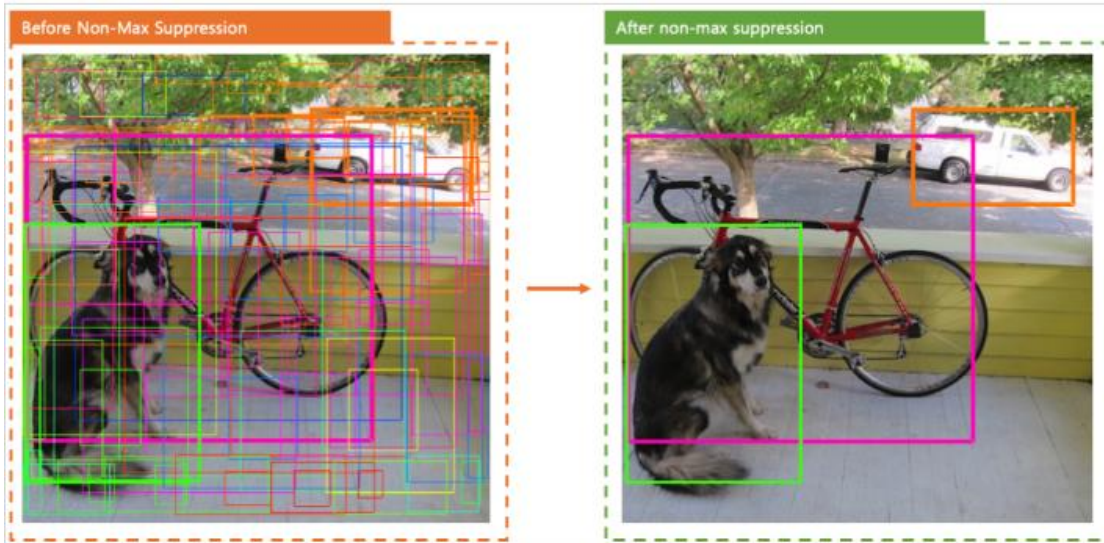
아웃풋 텐서의 하나의 그리드를 확대해 보면 총 크기 30의 아웃풋이 들어가 있다. 여기서 추론을 할 때는 class specific confidence score이라는 것을 계산하게 된다.

첫 번째 바운딩 박스의 confidence score와 각 클래스일 확률을 곱해 준다.



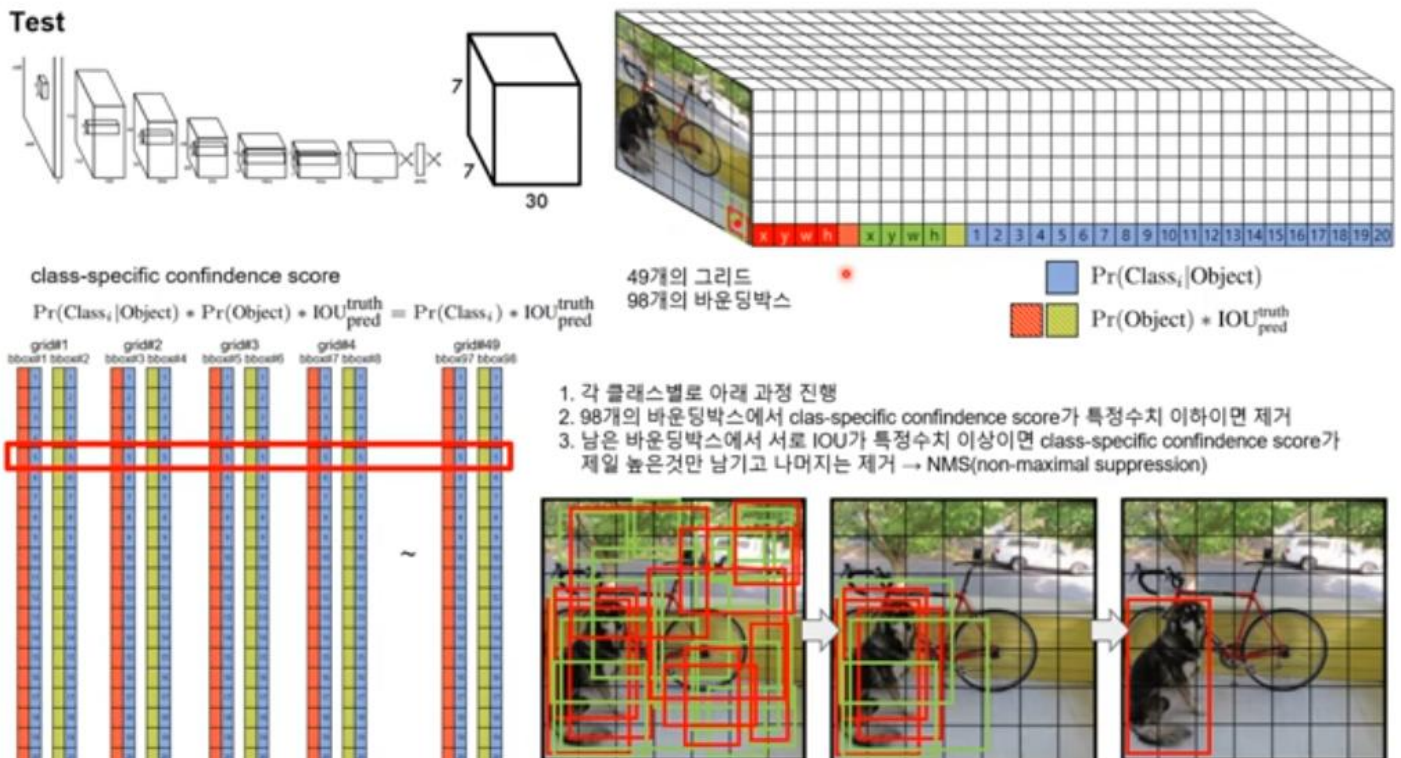
이 과정을 예측한 바운딩 박스의 개수만큼 32번 (16 cell 개수 x 2개 바운딩 박스) (위 동작원리 예시에서 가정한 개수) 해주게 된다. 그런데 여기까지 진행하게 되면 하나의 오브젝트 마다 바운딩 박스 개수가 너무 많아지게 된다. 이들 중 제일 확실한 하나를 고르기 위해 NMS라는 알고리즘을 사용하게 된다.

□ NMS (Non-Maximum Suppression)



< NMS 예시 사진 >

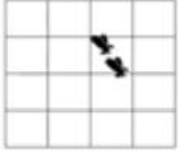
NMS란 object detection이 예측한 바운딩 박스들 중에서 가장 정확한 하나의 박스만 선택하도록 하는 알고리즘이다. 각 cell마다 confidence score 이 가장 큰 박스를 고르고 선택한 박스와의 IOU가 임계 값보다 큰 박스는 모두 제거한다. 하나의 박스만 남을 때까지 이 과정을 반복적으로 실행하는 것이다.



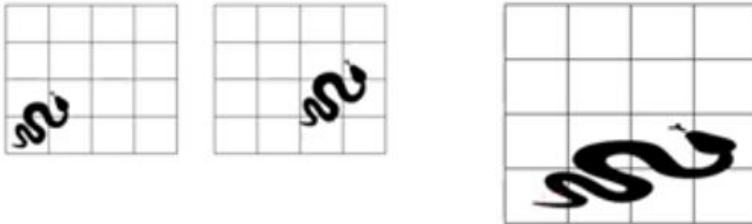
□ Limitation

YOLO의 한계점으로는 작은 물체에 대해 탐지 성능이 낮다는 것이다. object가 크면 bbox 간에 IOU 값의 차이가 커져서 적절한 predictor를 선택할 수 있지만, object가 작으면 bbox 간 IOU 값의 차이가 작아서 근소한 차이로 predictor가 결정되어 정확하지가 않다. 그리고 object 비율이 달라지면 detection 성능이 낮아진다는 것이다.

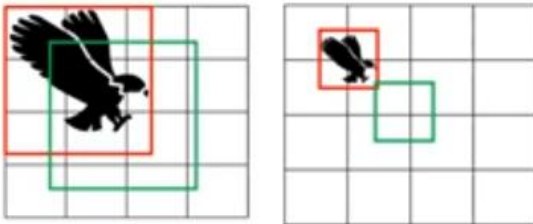
그리드셀당 하나의 객체만 탐지하기 때문에 작은 객체의 탐지가 어려움



데이터로부터 바운딩박스 예측을 학습하기 때문에 학습데이터에 없었던 종횡비는 대처가 어려움



바운딩박스의 크기에 상관없이 동일한 loss를 주기 때문에 성능에 악영향을 끼칠수 있음



참고 : YOLO 시리즈 (특히 YOLOv5~v8)는 PyTorch로 실습하는 게 훨씬 쉽고 완성도가 높다. TensorFlow에 익숙하더라도, YOLO만큼은 PyTorch 기반 ultralytics 라이브러리를 사용하는 걸 추천한다.

항목	PyTorch 기반 YOLO (ultralytics)	TensorFlow에서 YOLO
공식 지원	✅ YOLOv5~v8 전용	❌ 직접 변환하거나 비공식 모델 사용
설치 & 사용	<code>pip install ultralytics</code> 후 바로 사용 가능	복잡한 모델 로딩, 변환 필요
커뮤니티/문서	활발하고 풍부함	다소 제한적
사용 예제	이미지, 비디오, 웹캠 등 풍부한 코드 제공	대부분 PyTorch 기반 예제
모델 구조 변경	PyTorch에서는 자유롭고 빠름	TensorFlow로 직접 구현하려면 복잡

TensorFlow에 익숙한 사람이 PyTorch YOLO를 써도 될까?

문제 없다! 왜냐하면 ultralytics 라이브러리는 PyTorch를 몰라도 사용 가능할 정도로 API가 단순하다.

예: `results = model("이미지경로")` 이런 식으로 바로 사용 가능. 딥러닝 자체 흐름은 TensorFlow랑 거의 똑같고, 실습 위주면 PyTorch 문법은 거의 안 써도 된다.

목표	추천 방법
YOLO 구조 이해하고 실습하기	PyTorch 기반 <code>ultralytics</code> 사용
TensorFlow로 끝까지 해보고 싶다면	SSD, EfficientDet, Faster R-CNN 같은 모델 추천 (TF Hub 지원)
TensorFlow에서 YOLO 꼭 써야 한다면	<code>onnx → tf</code> 변환 또는 <code>openvino</code> 등 별도 도구 필요